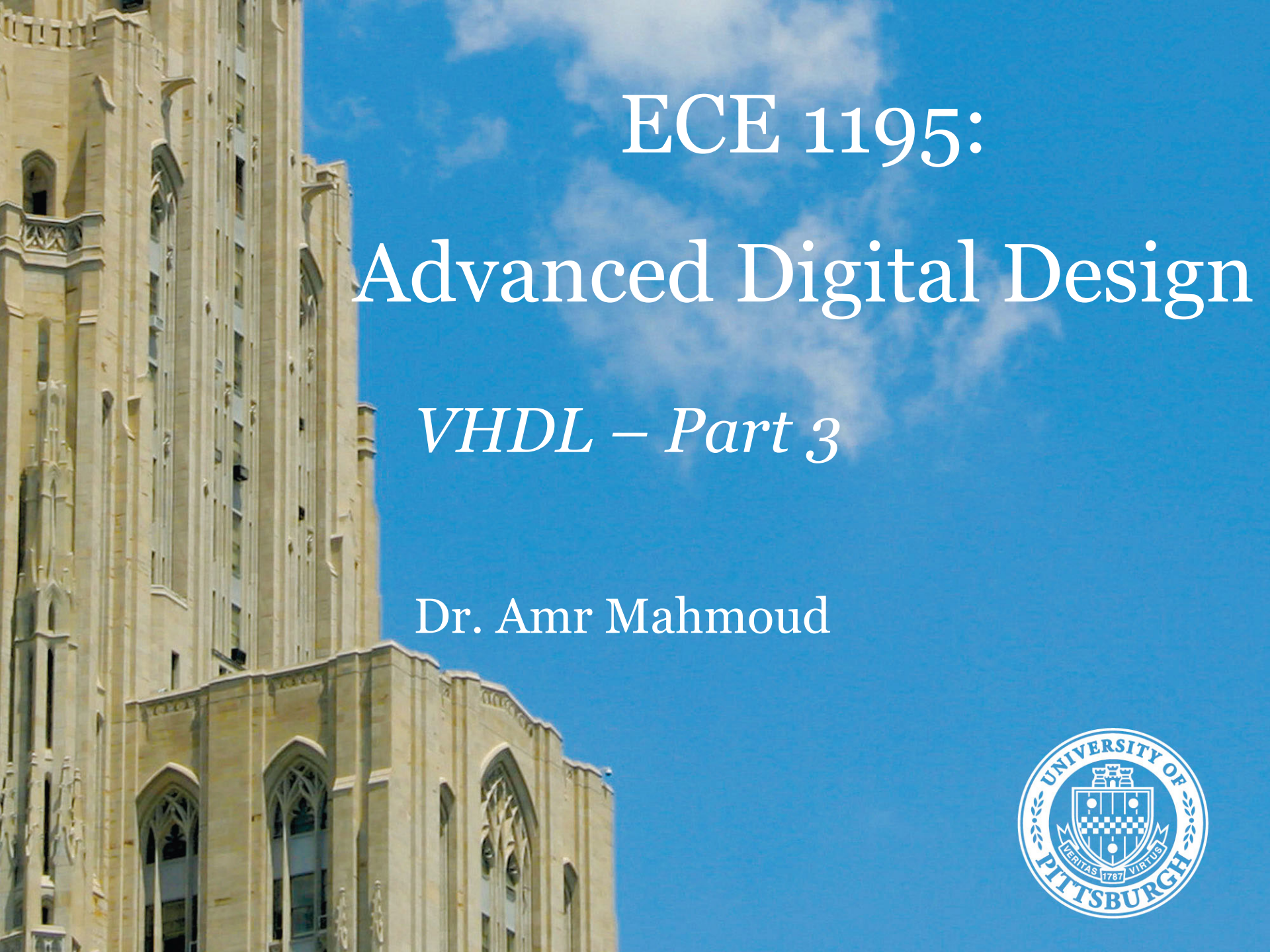


ECE 1195: Advanced Digital Design

VHDL – Part 3

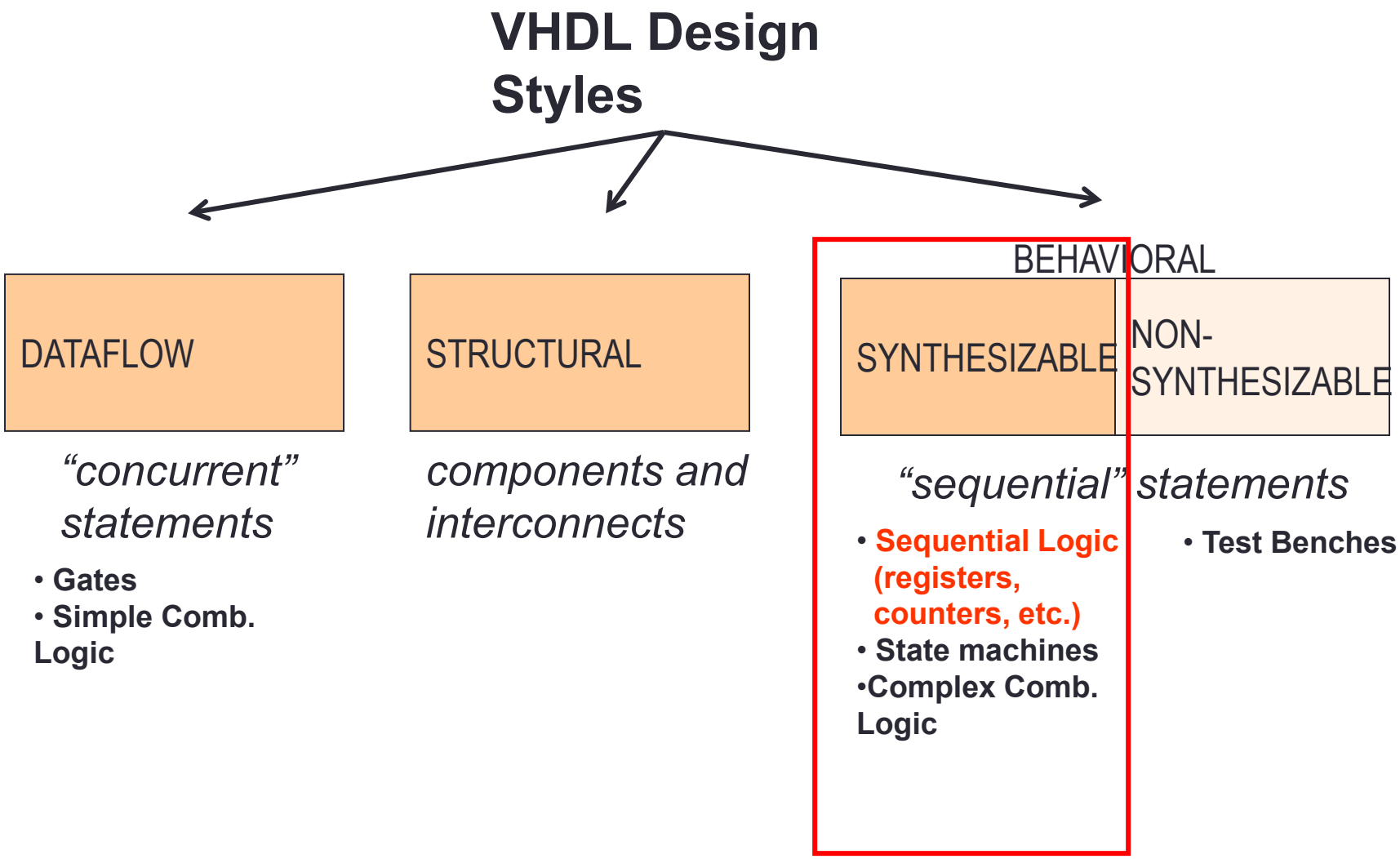
Dr. Amr Mahmoud

Behavioral VHDL Coding

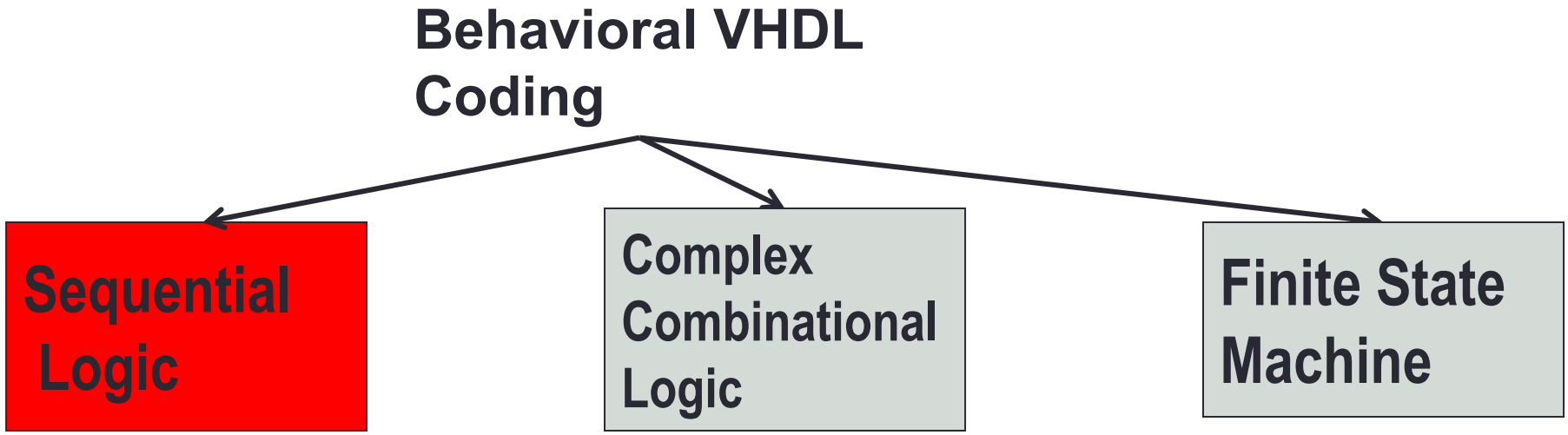
Agenda

- **Behavioral VHDL Coding for Synthesis**
  - Process Statement.
  - Sequential Logic: Latches, Flip-Flops, Registers, Shift Registers, Counters.
  - Behavioral VHDL Statements: If/Then, Wait, Case, For Loop, While Loop.

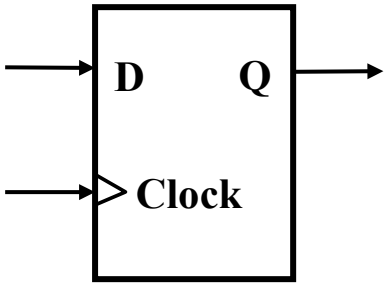
VHDL Design Styles



Behavioral VHDL Coding (for synthesis) uses



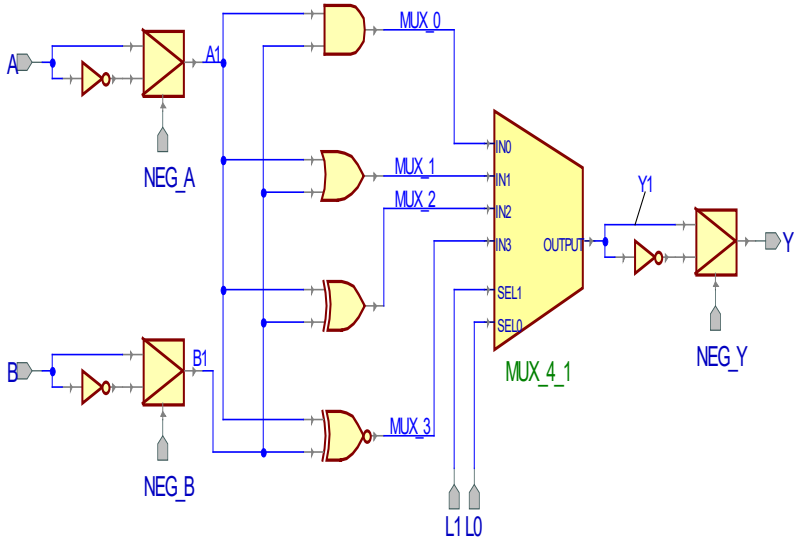
- (registers, counters, etc.)



Behavioral VHDL Coding (for synthesis) uses

Behavioral VHDL Coding

- **Sequential Logic**
- **Complex Combinational Logic**
- **Finite State Machine**

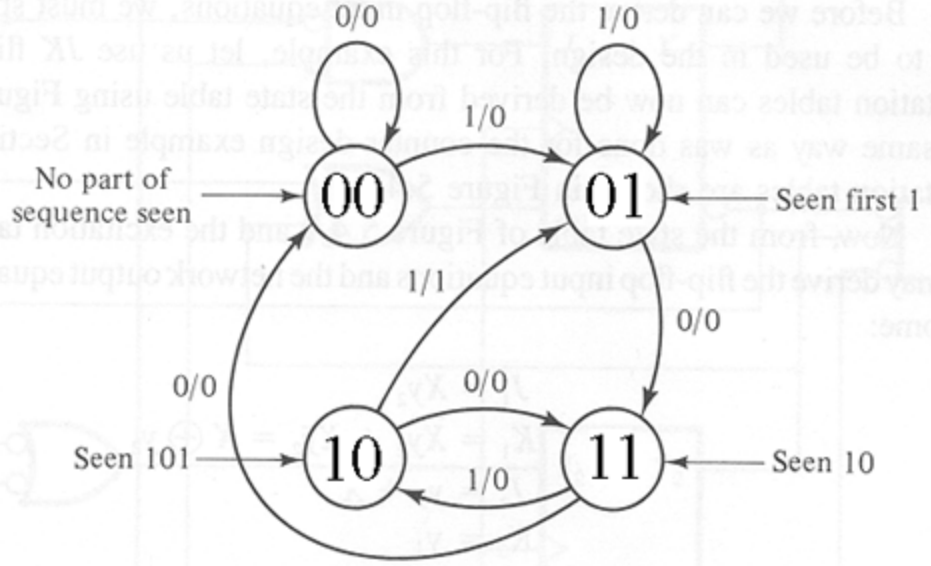


you should be Careful

Behavioral VHDL Coding (for synthesis) uses

Behavioral VHDL Coding

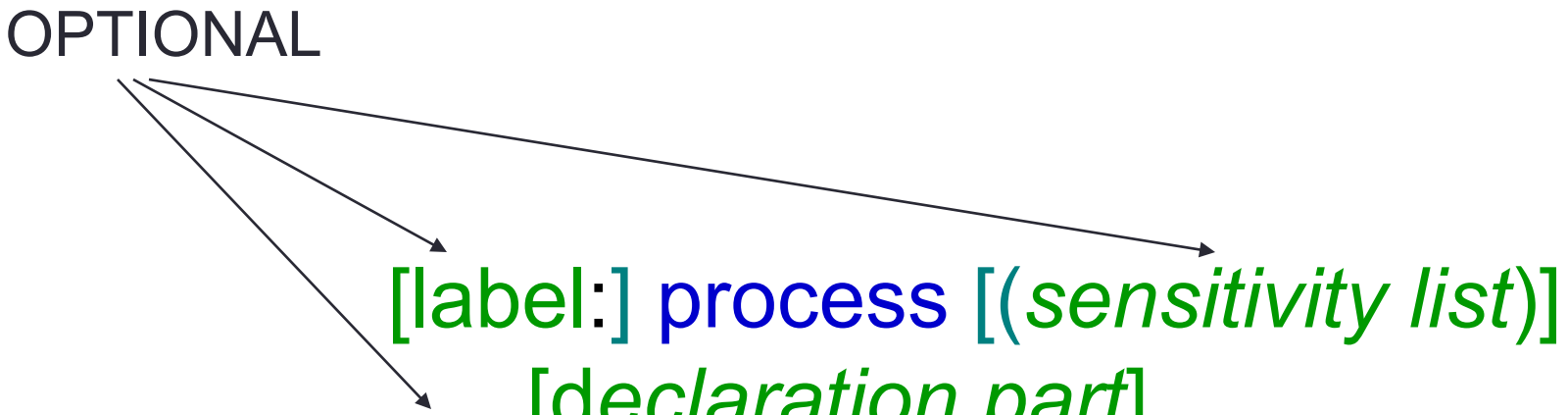
- **Sequential Logic**
- **Complex Combinational Logic**
- **Finite State Machine**



Behavioral Code

- Behavioral code also referred to as "sequential code"
- Behavioral code is characterized by **PROCESS STATEMENTS**
  - In process statements, sections of code are executed sequentially—order matters!

Anatomy of a Process



Sensitivity List

- List of signals to which the process is sensitive.
  - Whenever there is an event on any of the signals in the sensitivity list, the process gets triggered.
  - **If an event occurs on a signal not listed in the sensitivity list, process does not get triggered**
- Every time the process gets triggered, it will run in its entirety.
- When designing synthesizable code, use a sensitivity list

```
process (sensitivity list)
    declaration part
begin
    statement part
end process;
```

- **Processes are executed Whenever there is an event on any of the signals in the sensitivity list**

Sensitivity List (Cont.)

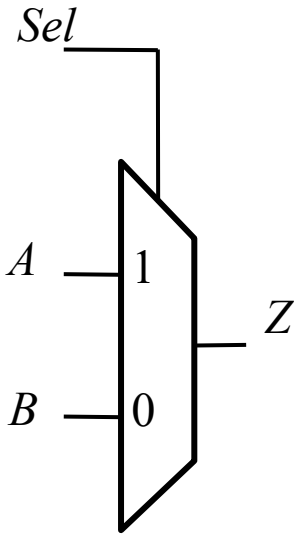
- Process without a sensitivity list is used in a **test bench**
  - Usually not synthesizable
  - It will start at the beginning of simulation
  - When it finishes it will restart at beginning of process (unless **WAIT** command issued)

```
process
declaration part
begin
    statement part
end process;
```

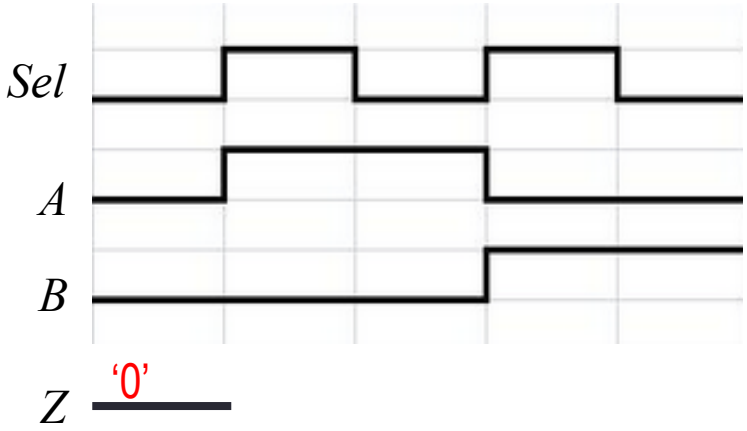
Processes (without Sensitivity List) are executed endlessly unless broken by WAIT statement

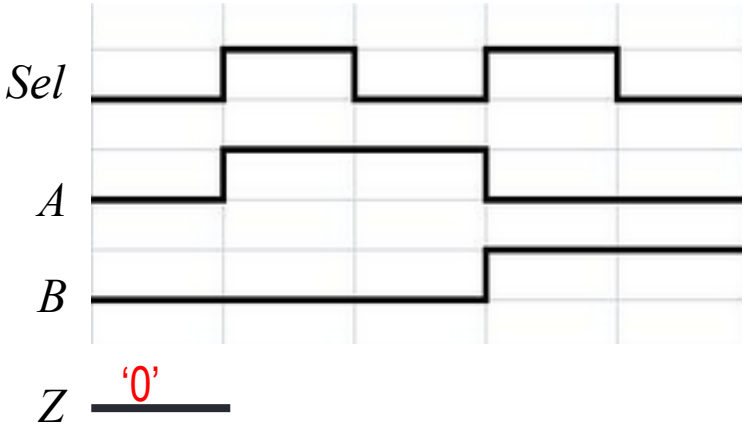
Example #1: Mux2to1

```
process (A ,B ,Sel)
begin
    IF(Sel = '1') Then
          Z <=A;
    ELSE
          Z <= B;
    END IF;
end process;
```



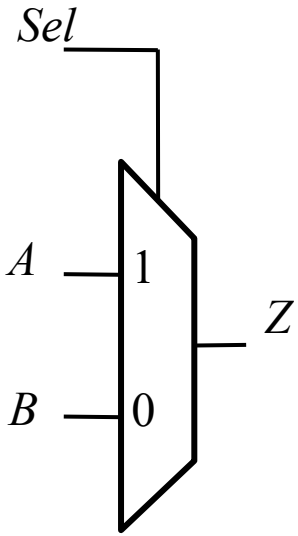

```
process (A ,B)
begin
    IF(Sel = '1') Then
          Z <=A;
    ELSE
          Z <= B;
    END IF;
end process;
```



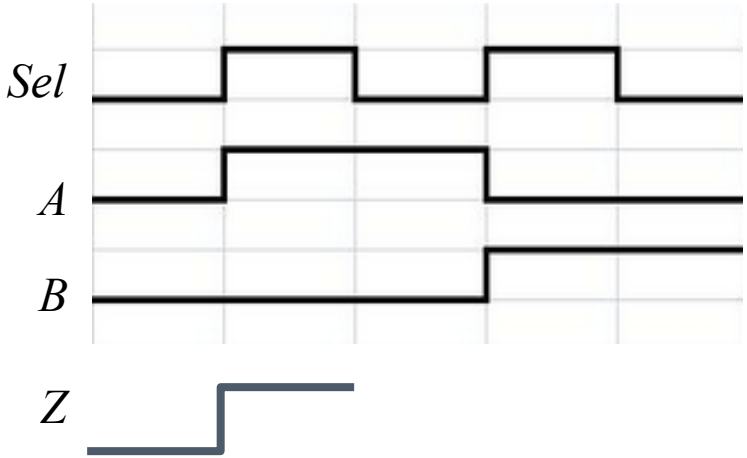


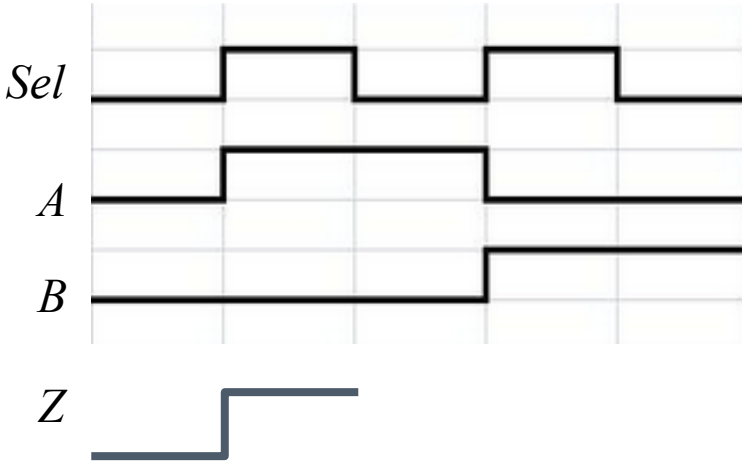
Example #1: Mux2to1

```
process (A ,B ,Sel)
begin
    IF(Sel = '1') Then
        Z <=A;
    ELSE
        Z <= B;
    END IF;
end process;
```



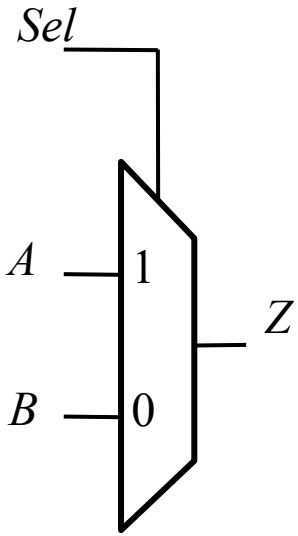

```
process (A ,B)
begin
    IF(Sel = '1') Then
        Z <=A;
    ELSE
        Z <= B;
    END IF;
end process;
```



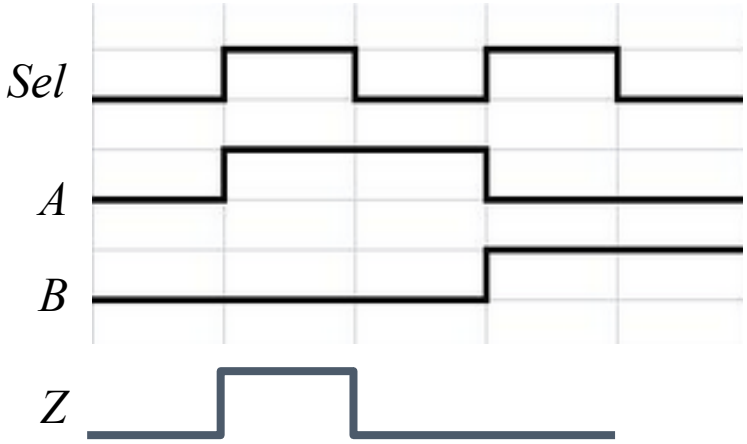


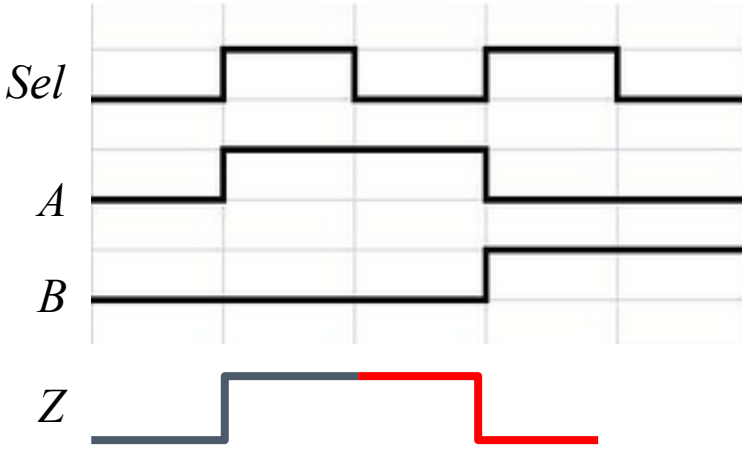
Example #1: Mux2to1

```
process (A ,B ,Sel)
begin
    IF(Sel = '1') Then
        Z <=A;
    ELSE
        Z <= B;
    END IF;
end process;
```




```
process (A ,B)
begin
    IF(Sel = '1') Then
        Z <=A;
    ELSE
        Z <= B;
    END IF;
end process;
```





Declaration Part

- Declaration part (optional)
  - Lists **variables** of the process; **signals** are declared in architecture declaration.

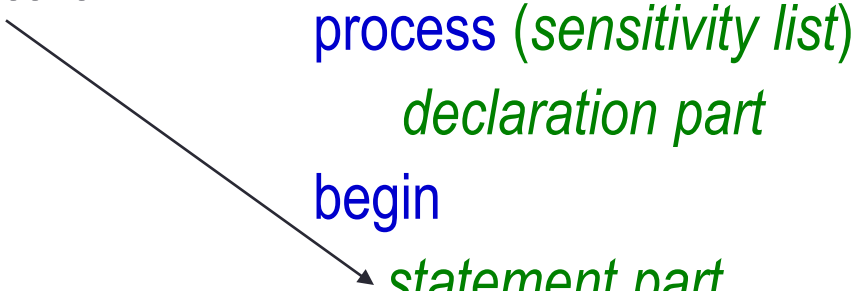
Variable	Signal
Variables exist only locally in a process	Signals exist globally in a architecture
USE :=	USE <=
Immediately Updated	Updated at the end of the Process

```
process (sensitivity list)
    declaration part
begin
    statement part
end process;
```

To get variable's value out of process, need to assign to signal

Statement Part

- Statement Part
  - Contains sequential statements to be executed each time the process Is activated
- Analogous to conventional programming languages



```
process (sensitivity list)
    declaration part
begin
    statement part
end process;
```

WITHIN THE STATEMENT PART, THE SEQUENCE OF OPERATIONS MATTERS!!

Processes in VHDL

- Processes Describe Sequential Behavior
- Processes in VHDL Are Very Powerful Statements
  - Allow to define an arbitrary behavior that may be difficult to represent by a real circuit
  - Not every process can be synthesized
- **Use Processes with Caution in the Code to Be Synthesized**

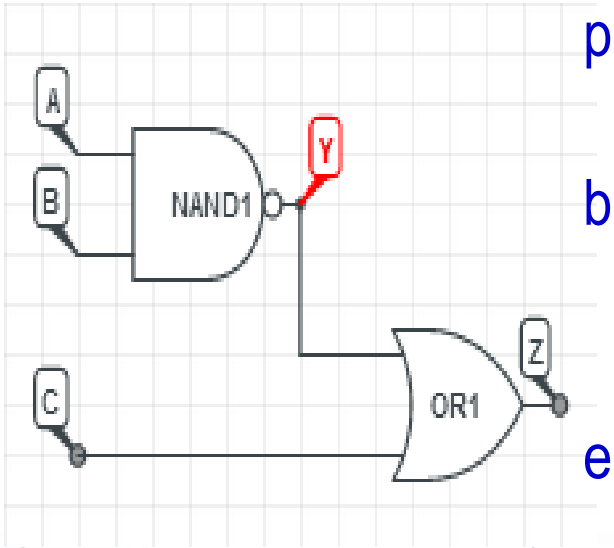
Processes in VHDL(Cont.)

- An Architecture can have multiple processes.
- All processes execute concurrently.

```
L1: process (sensitivity list)
      declaration part
begin
     statement part
end process;
L2: process (sensitivity list)
      declaration part
begin
     statement part
end process;
```

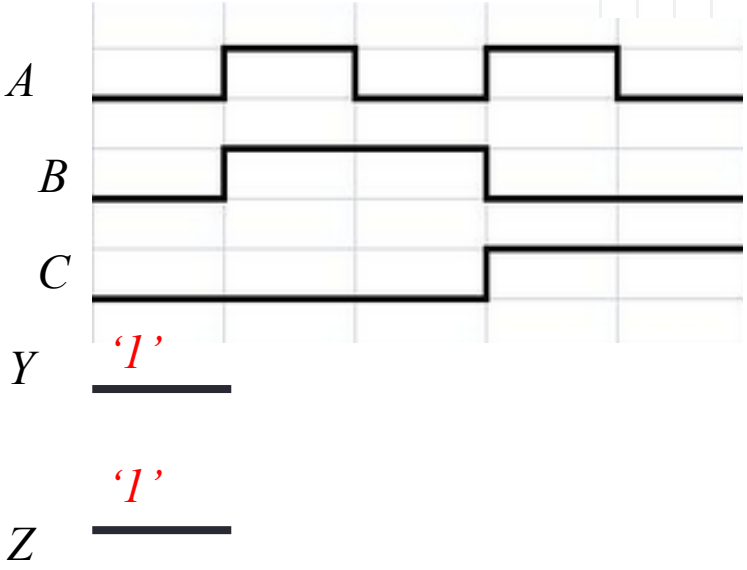
Example #2: Logic Gates

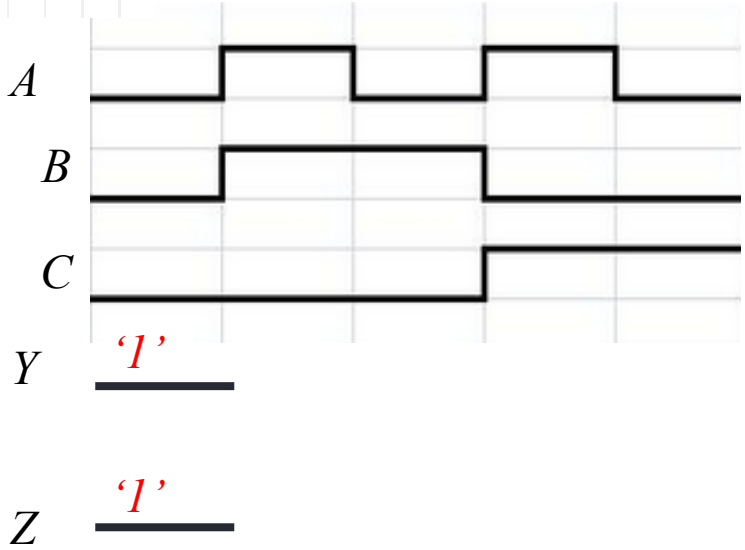
```
process (A,B,C)
Variable y: STD_LOGIC;
begin
    y :=A Nand B;
    Z <= y OR C;
end process;
```




```
process (A,B,C)

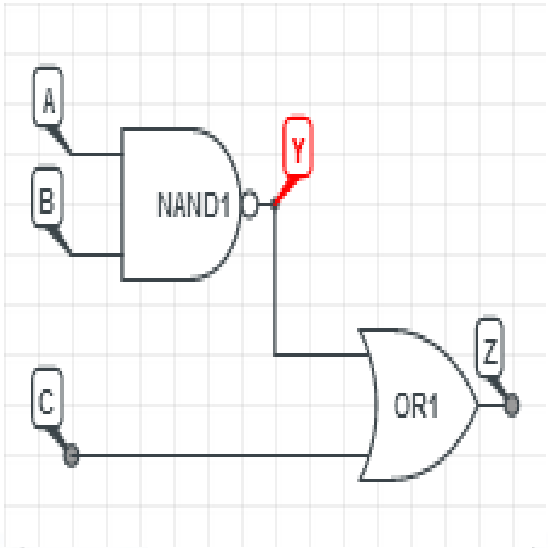
begin
    y <=A Nand B;
    Z <= y OR C;
end process;
```





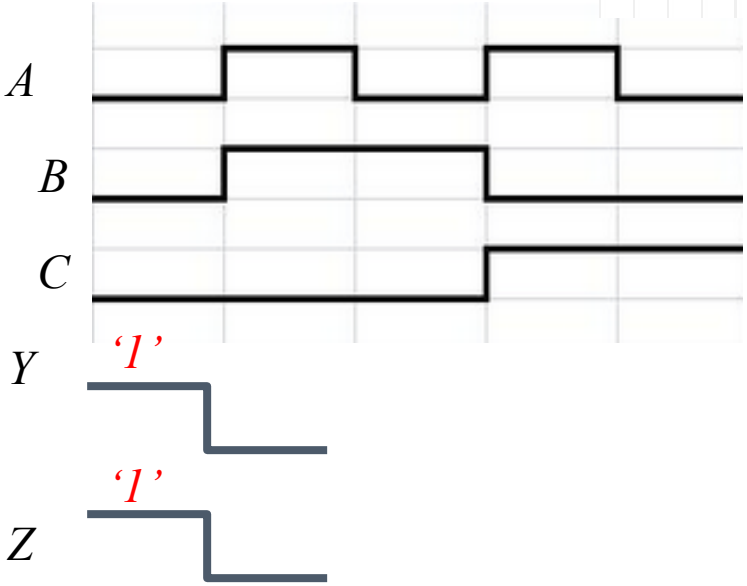
Example #2: Logic Gates

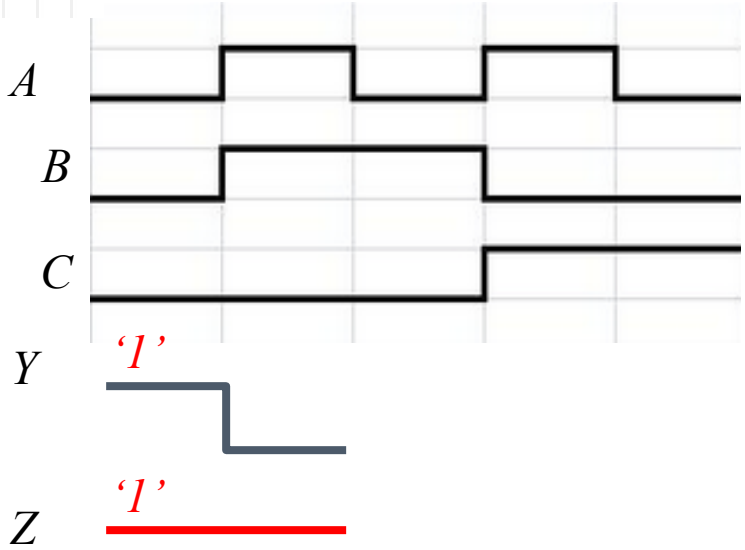
```
process (A,B,C)
Variable y: STD_LOGIC;
begin
    y :=A Nand B;
    Z <= y OR C;
end process;
```




```
process (A,B,C)

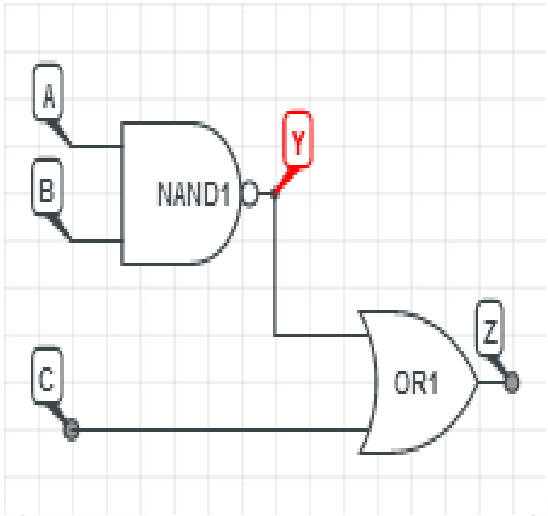
begin
    y <=A Nand B;
    Z <= y OR C;
end process;
```





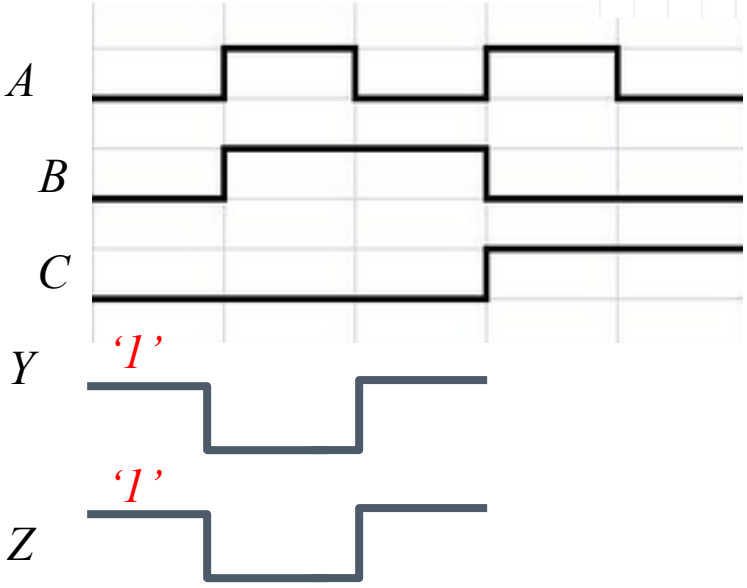
Example #2: Logic Gates

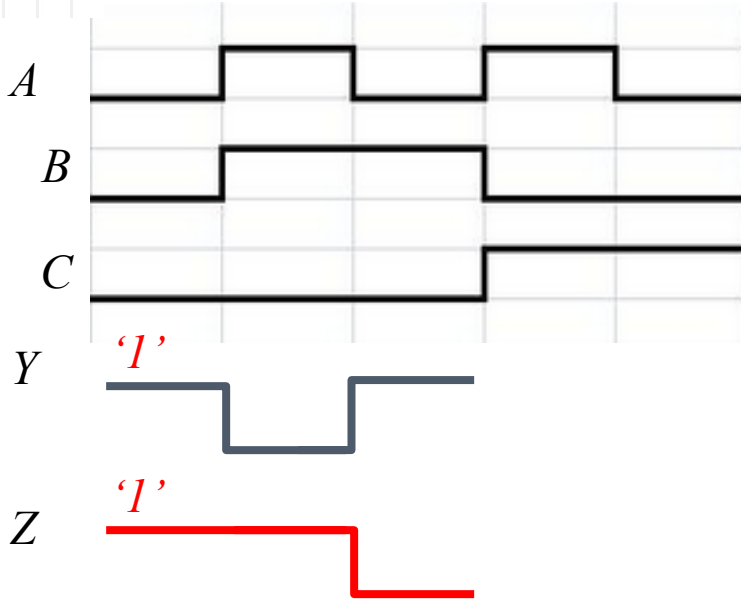
```
process (A,B,C)
Variable y: STD_LOGIC;
begin
    y :=A Nand B;
    Z <= y OR C;
end process;
```




```
process (A,B,C)

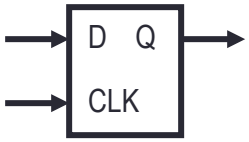
begin
    y <=A Nand B;
    Z <= y OR C;
end process;
```

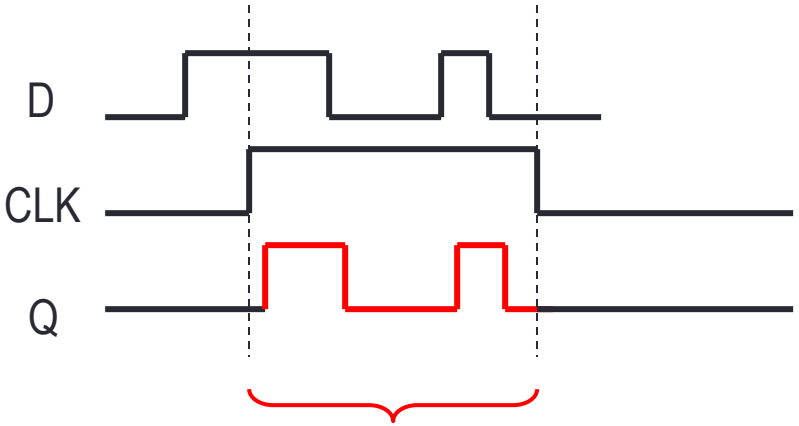




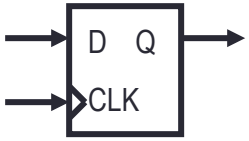
SEQUENTIAL LOGIC
(LATCHES, FLIP-FLOPS, REGISTERS)

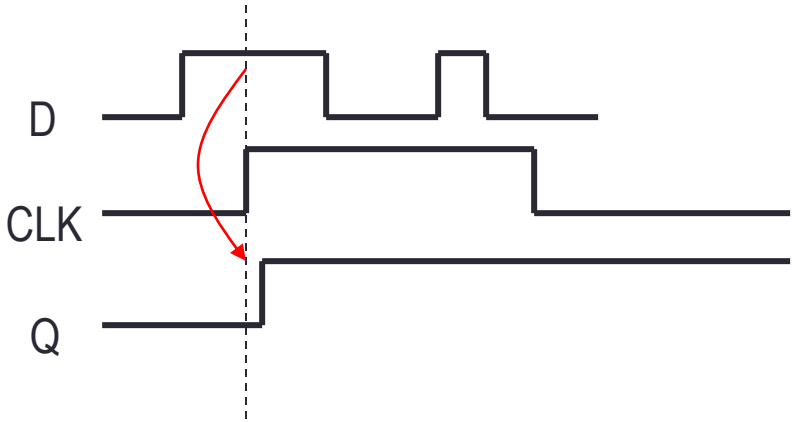
D Latch vs. D Flip-Flop





Latch transparent when clock is high





"Samples" D on rising edge of clock

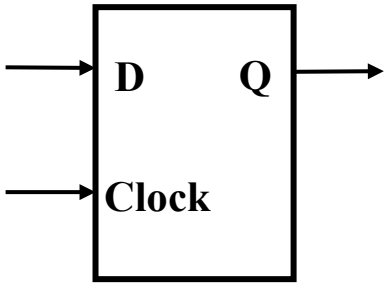
Sensitivity List ????

D Latch

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY latch IS
    PORT (          D, Clock  : IN STD_LOGIC ;
            Q                 : OUT STD_LOGIC) ;
END latch ;

ARCHITECTURE Behavior OF latch IS
BEGIN
    PROCESS ( D, Clock )
    BEGIN
        IF Clock = '1' THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END Behavior;
```



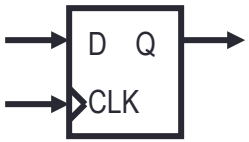
Process is sensitive to D and clock

D Flip-Flop: Edge Sensitive

- **How to check when the edge comes !!!!**
  - Positive(rising) Edge
    - Clock'Event AND Clock ='1'
    - Rising_Edge(Clock)

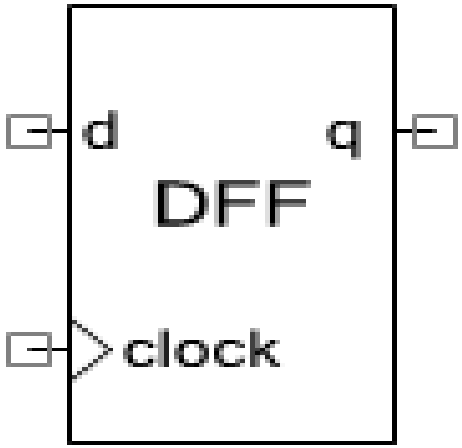
```
IF (Clock'EVENT AND Clock = '1')
THEN
        Q <= D ;
END IF ;
```

  - Negative(falling) Edge
    - Clock'Event AND Clock ='0'
    - Falling_Edge(Clock)



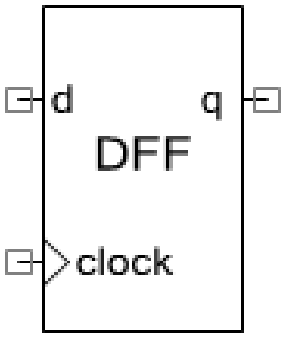

```
IF (Clock'EVENT AND Clock = '0')
THEN
        Q <= D ;
END IF ;
```

- Write the VHDL code that describe **D-Flip Flop**:



Solution

- Write the VHDL code that describe **D- Flip Flop**:



Equivalent to
rising_edge(Clock)

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY flipflop IS
  PORT (  D, Clock : IN STD_LOGIC ;
          Q    : OUT STD_LOGIC) ;
END flipflop ;

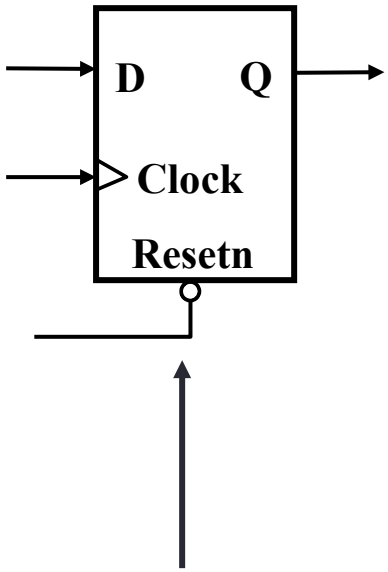
ARCHITECTURE Behavior_1 OF flipflop IS
BEGIN
  PROCESS ( Clock )
  BEGIN
    IF (Clock'EVENT AND Clock = '1') THEN
      Q <= D ;
    END IF ;
  END PROCESS ;
END Behavior_1 ;
```

D flip-flop with asynchronous reset

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

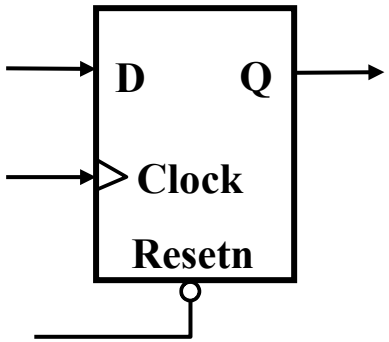
ENTITY flipflop IS
    PORT ( D, Resetn, Clock : IN  STD_LOGIC ;
           Q                : OUT  STD_LOGIC) ;
END flipflop ;

ARCHITECTURE Behavior OF flipflop IS
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            Q <= '0' ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END Behavior ;
```



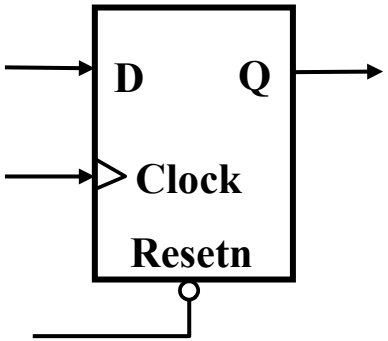
Bubble means active-low reset (written resetn for reset "not")

- Write the VHDL code that describe **D-Flip Flop with Synchronous Reset**:



Solution

- Write the VHDL code that describe **D-Flip Flop with Synchronous Reset** :




```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY SynFlip IS
  PORT (  D, Resetn, Clock  : IN  STD_LOGIC ;
    Q   : OUT  STD_LOGIC) ;
END SynFlip ;

ARCHITECTURE Behavior OF SynFlip IS
BEGIN
  PROCESS (Clock)
  BEGIN
    IF (Clock'EVENT AND Clock = '1') THEN
      IF (Resetn = '0') THEN
        Q <= '0' ;
      ELSE
        Q <= D ;
      END IF ;
    END IF ;
  END PROCESS ;
END Behavior ;
```

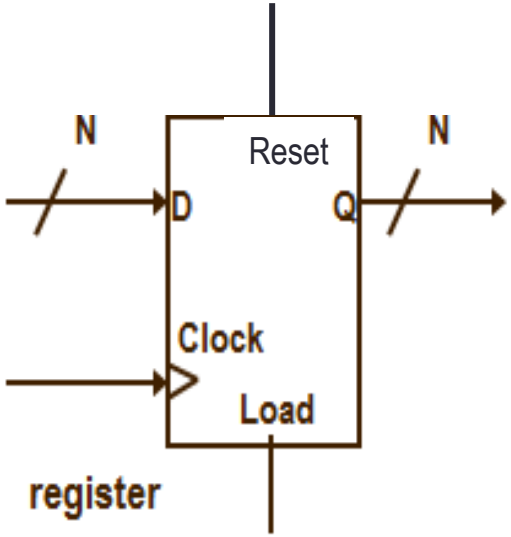
Asychronous vs. Synchronous

- In the IF statement, **asynchronous items** are
  - Before the Clock'EVENT AND Clock = '1' statement
  - Written in the Sensitivity List.

- In the IF statement, **synchronous items** are
  - After the Clock'EVENT AND Clock = '1' statement
  - **Not** written in the Sensitivity List

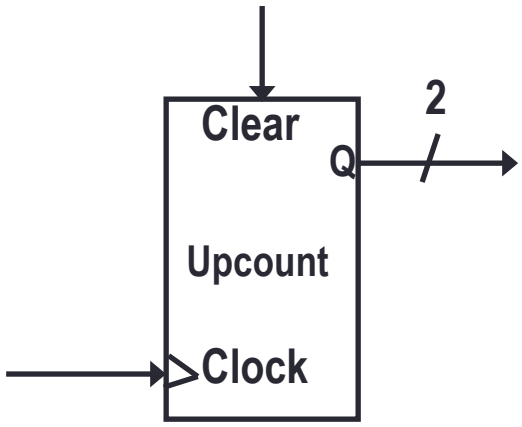
Lab 3 Register

- Write the VHDL code that describes **N-bit register with synchronous Load enable and asynchronous reset**



COUNTERS

2-bit up-counter with synchronous reset



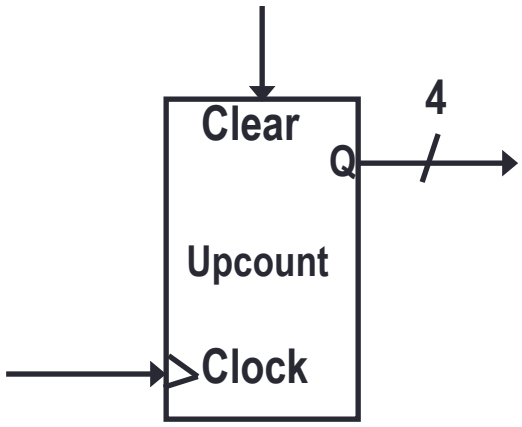

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_unsigned.all ;

ENTITY Upcount2 IS
  PORT ( Clear, Clock : IN   STD_LOGIC ;
         Q     : OUT  STD_LOGIC_VECTOR(1 DOWNTO 0) ) ;
END Upcount2;

ARCHITECTURE Behavior OF Upcount2 IS
SIGNAL Count : std_logic_vector(1 downto 0);
BEGIN
upcount: PROCESS ( Clock )
  BEGIN
      IF (Clock'EVENT AND Clock = '1') THEN
         IF (Clear = '1') THEN
                  Count <= "00" ;
         ELSE
                  Count <= Count + 1 ;
         END IF ;
      END IF;
  END PROCESS;
Q <= Count;
END Behavior ;
```

Assigned outside the process →

4-bit up-counter with Asynchronous reset

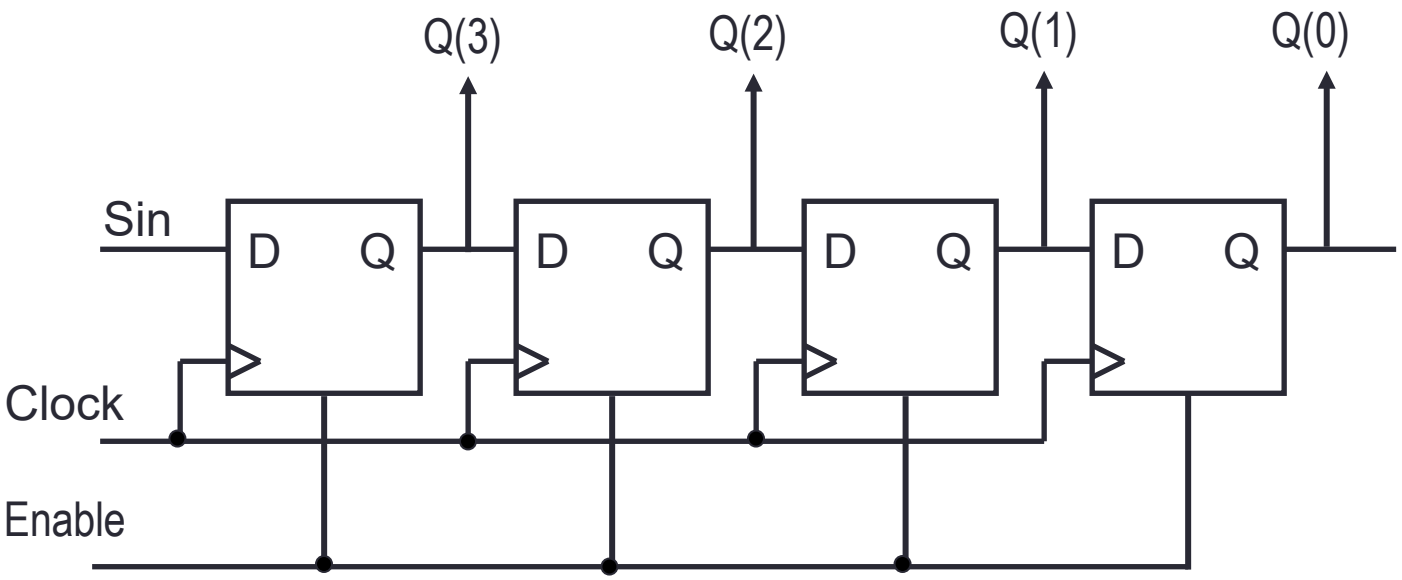



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_unsigned.all ;
-------------------------------------------------
ENTITY Asyn_upcount IS
  PORT (  Clock, Resetn, Enable : IN STD_LOGIC ;
      Q       : OUT STD_LOGIC_VECTOR (3 DOWNTO 0)) ;
END Asyn_upcount ;
-------------------------------------------------
ARCHITECTURE Behavior OF Asyn_upcount IS
  SIGNAL Count : STD_LOGIC_VECTOR (3 DOWNTO 0) ;
BEGIN
  PROCESS ( Clock, Resetn )
  BEGIN
    IF Resetn = '0' THEN
      Count <= "0000" ;
    ELSIF (Clock'EVENT AND Clock = '1') THEN
      IF (Enable = '1') THEN
        Count <= Count + 1 ;
      END IF ;
    END IF ;
  END PROCESS ;
  Q <= Count ;
END Behavior ;
```

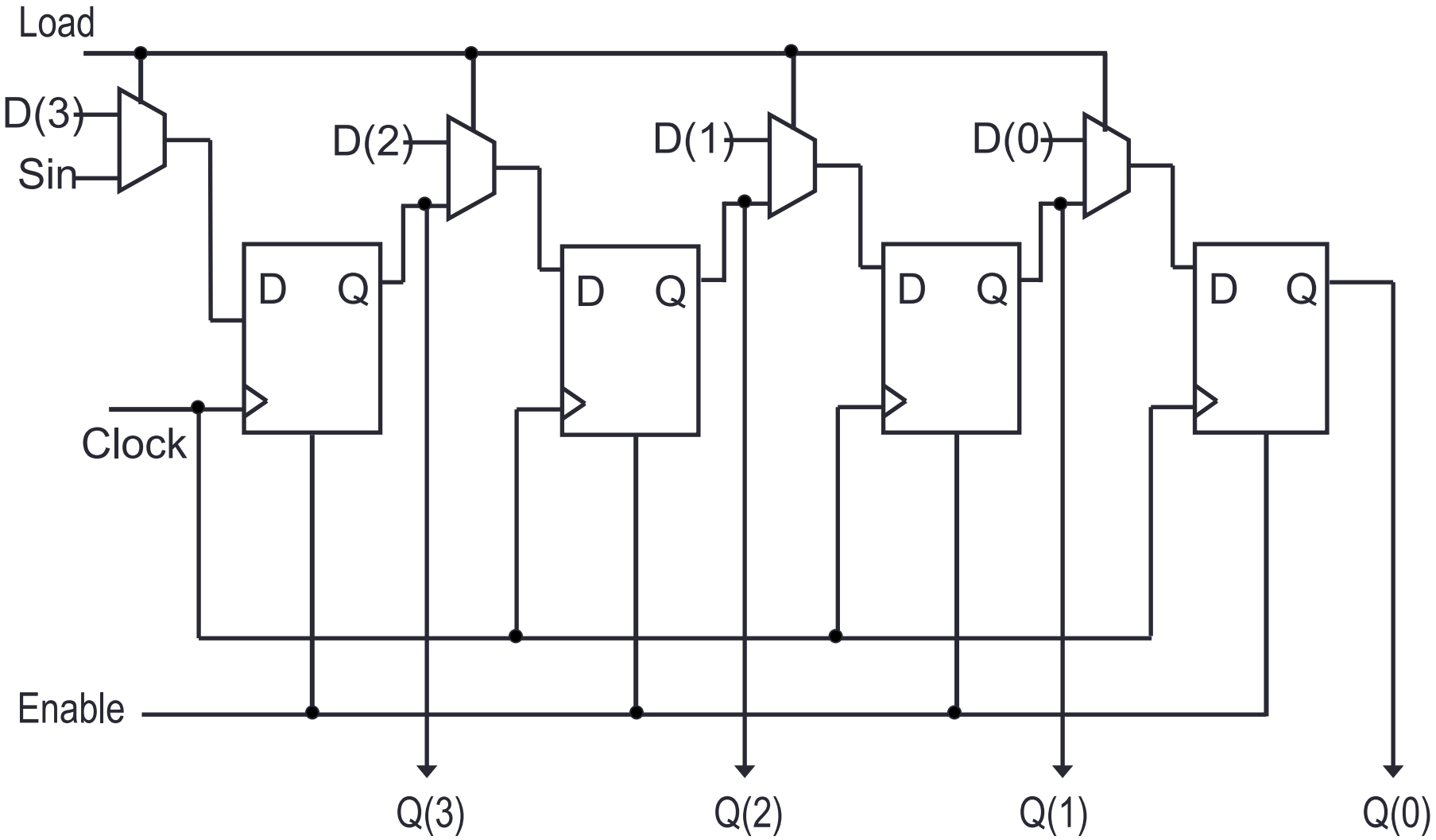
Assigned outside the process →

SHIFT REGISTERS

Shift Register

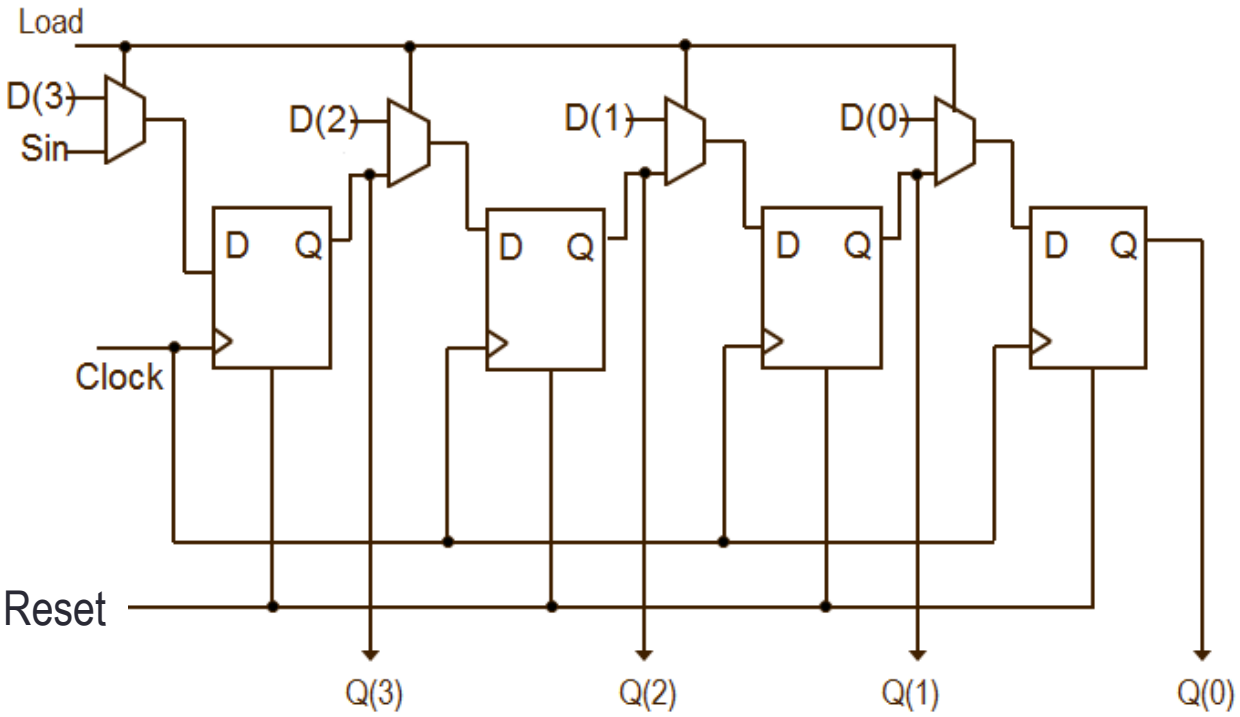


Shift Register With Parallel Load



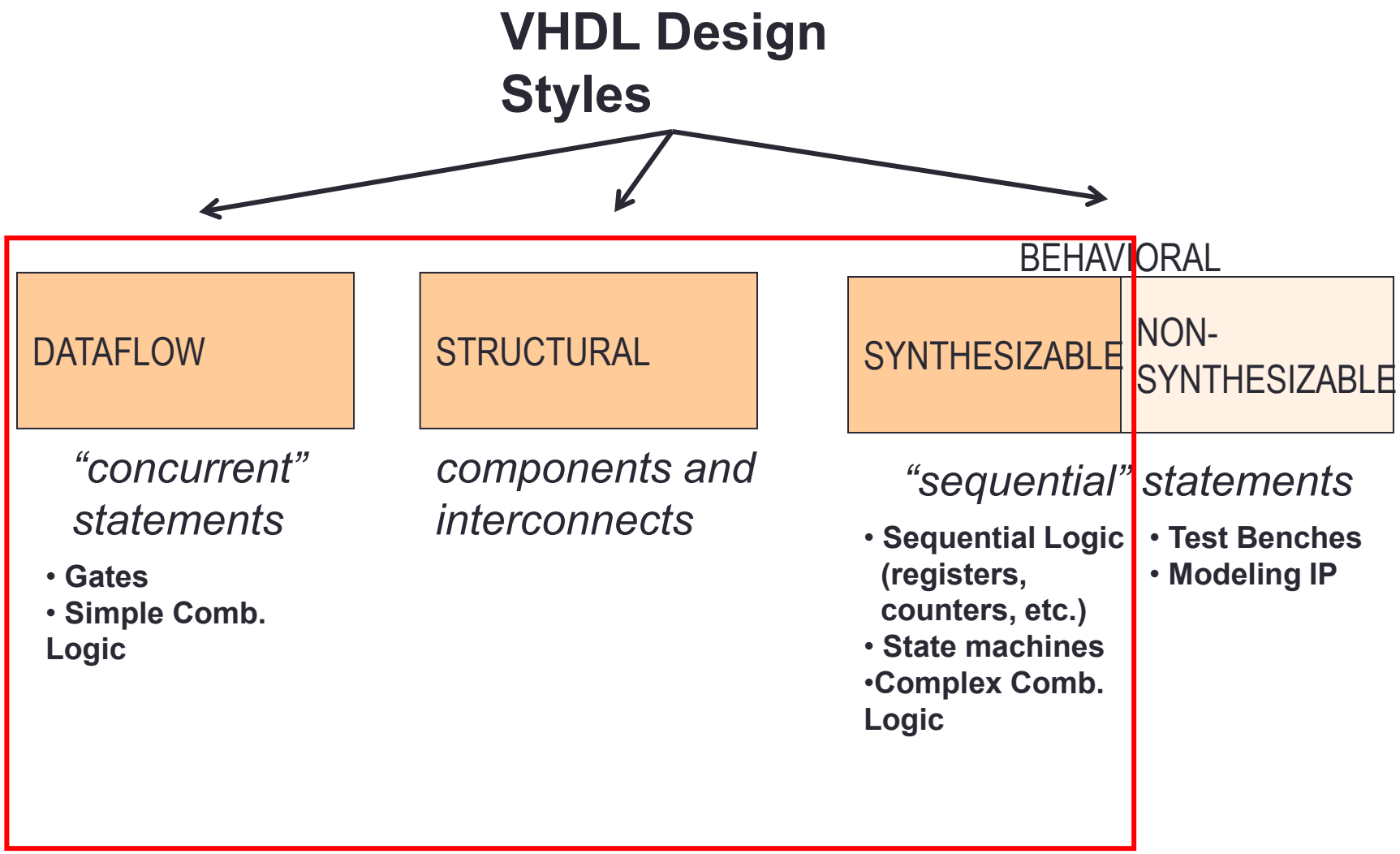
Lab 3 Shift Register

- Write the VHDL code that describes **N-bit Shift left/right register with synchronous Parallel Load and asynchronous reset**



MIXING DESIGN STYLES IN AN ARCHITECTURE

VHDL Design Styles (Architecture)



Mixed Style Modeling

```
architecture ARCHITECTURE_NAME of ENTITY_NAME is
```

- Here you can declare signals, constants, functions, procedures…
- Component declarations
- **No variable declarations !!**

```
begin
```

Concurrent statements:

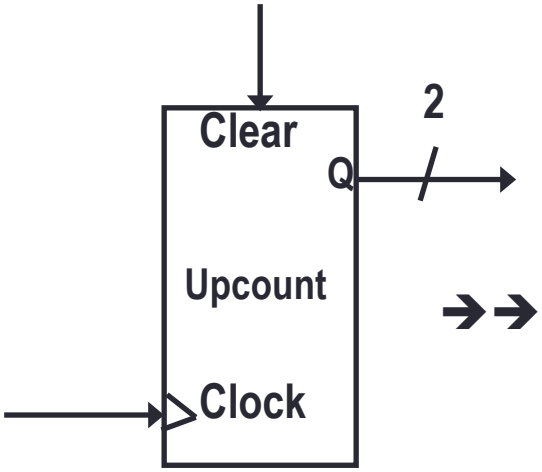
- Concurrent simple signal assignment
- Conditional signal assignment
- Selected signal assignment
- Generate statement
- Component instantiation statement
- Process statement
  - **inside process you can use only sequential statements**

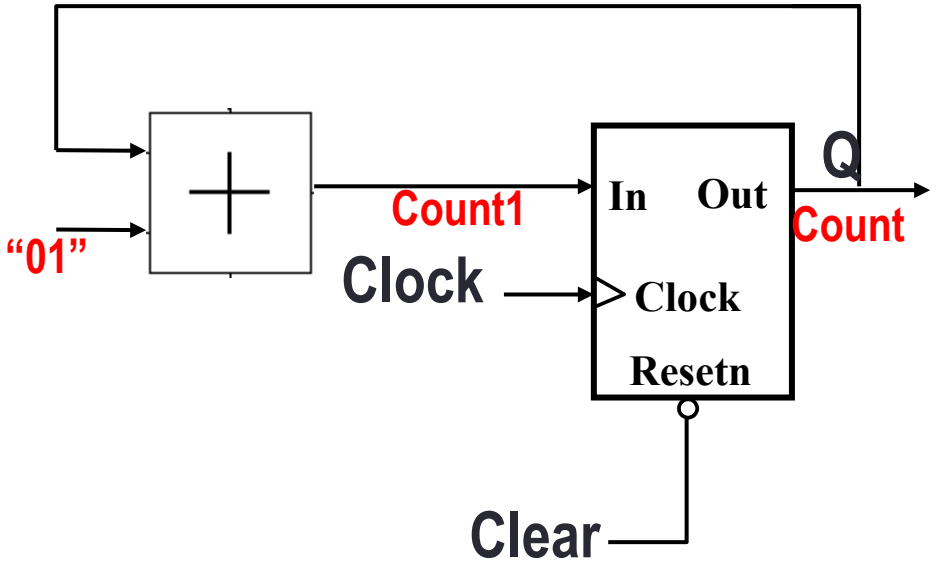
Concurrent Statements

(i.e. two processes are concurrent)

```
end ARCHITECTURE_NAME;
```

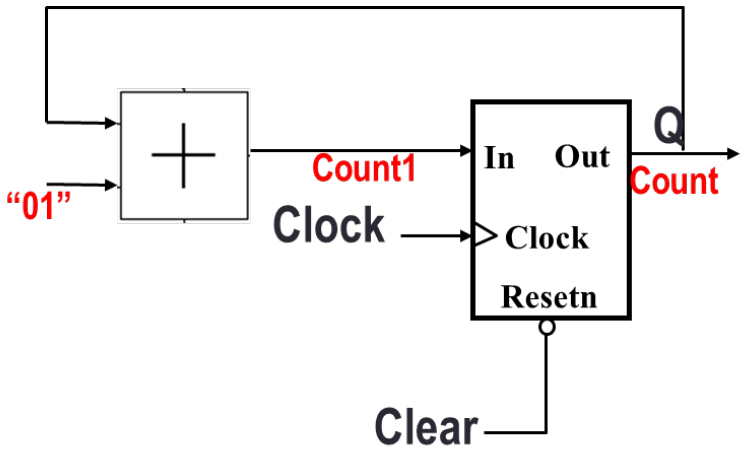
- Write the VHDL code that describe **2-bit up-counter with synchronous reset** **without including any arithmetic library (don't use "+") (assume you have n-bit adder)**





Solution

- Write the VHDL code that describe **2-bit up-counter with synchronous reset** **without including any arithmetic library (don't use "+")**



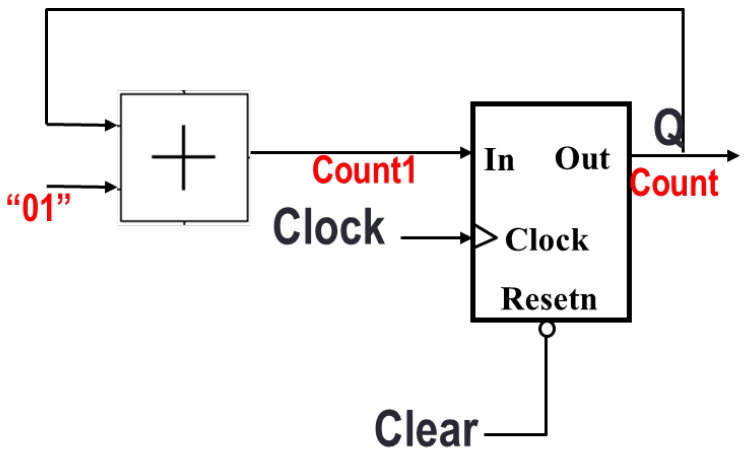

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY upcount IS
  PORT ( Clear, Clock : IN    STD_LOGIC ;
         Q    : OUT  STD_LOGIC_VECTOR(1 DOWNTO 0) ) ;
END upcount ;

ARCHITECTURE Behavior OF upcount IS
--------------------
SIGNAL Count,Count1 : std_logic_vector(1 downto 0);
SIGNAL Temp : std_logic;
COMPONENT Nbit_adder IS
     GENERIC(N: Integer := 4);
     PORT(A: IN STD_LOGIC_VECTOR(N-1 downto 0);
          B: IN STD_LOGIC_VECTOR(N-1 downto 0);
          Cin: IN STD_LOGIC;
          Sum: OUT STd_LOGIC_VECTOR(N-1 downto 0);
          Cout: OUT STD_LOGIC
          );
END COMPONENT;
--------------------
```

Solution

- Write the VHDL code that describe **2-bit up-counter with synchronous reset** **without including any arithmetic library (don't use "+")**




```
-------------------------------------
BEGIN
U1: Nbit_adder
      GENERIC MAP(N =>2)
      PORT MAP ( A=> Count,
                 B=> "01",
                 Cin=> '0',
                 Sum=> Count1,
                 Cout => Temp);
upcount: PROCESS ( Clock)
  BEGIN
    IF (Clock'EVENT AND Clock = '1') THEN
      IF (Clear = '1') THEN
        Count <= "00" ;
      ELSE
        Count <= Count1;
      END IF ;
    END IF;
END PROCESS;
-----------------------------------------
Q <= Count;
END Behavior ;
```

BEHAVIORAL VHDL STATEMENTS

If Statement

- IF statement

if *boolean expression* then
    statements
elsif *boolean expression* then
    statements
    ⋮
else *boolean expression* then
    statements
end if;

```
 SELECTOR: process (Clock)
 begin
     IF (Clock'EVENT AND Clock = '1')
THEN
   IF Sel = "00" THEN
    f  <= x1;
   ELSIF Sel = "10" THEN
    f  <= x2;
   ELSE
    f  <= x3;
   END IF;
      END IF;
 end process;
```

- else and elsif are optional
- Most commonly used type of statement

Equivalent to " **WHEN ELSE** " statement.

Case Statement

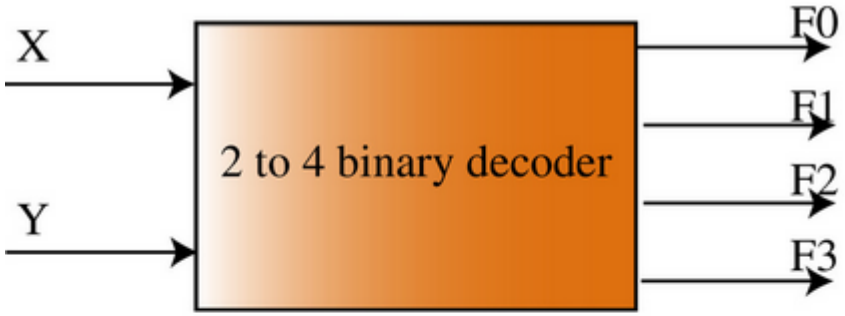
- Case statement
- Choices have to cover all possible values of the expression
  - Use others to specify all remaining cases

```
case expression is
    when choice_1 =>
        statements
    when choice_2 =>
        statements
        ⋮

    when others =>
        statements
end case;
```

Equivalent to " **WITH-SELECT-WHEN**" statement.

- Write the VHDL code that describe **2-to-4 decoder using Case Statement**



X	Y	F0	F1	F2	F3
0	0	1	0	0	0
1	0	0	1	0	0
0	1	0	0	1	0
1	1	0	0	0	1

Solution

- Write the VHDL code that describe **2-to-4 decoder using Case Statement**

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY Dec2to4 IS
  PORT ( A : IN STD_LOGIC_VECTOR(1 DOWNTO 0) ;
         Y : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)) ;
END Dec2to4 ;
ARCHITECTURE Behavior OF Dec2to4 IS
BEGIN
  PROCESS (A)
  BEGIN
    Case(A) IS
        WHEN "00" =>
                 Y<= "0001";
        WHEN "01" =>
                 Y<= "0010";
        WHEN "10" =>
                 Y<= "0100";
        WHEN others =>
                 Y<= "1000";
    END Case;
   END PROCESS;
END Behavior ;
```

For Loop

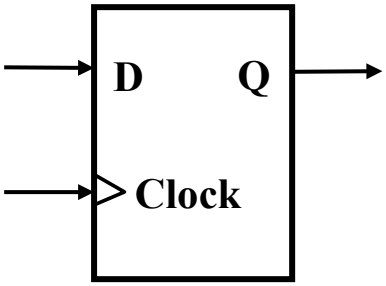
- For Loop Statement

```
[label:] for i in range loop
        statements
end loop [label];
```

- Repeats a section of VHDL code sequentially (i.e. order matters)

Wait Statements

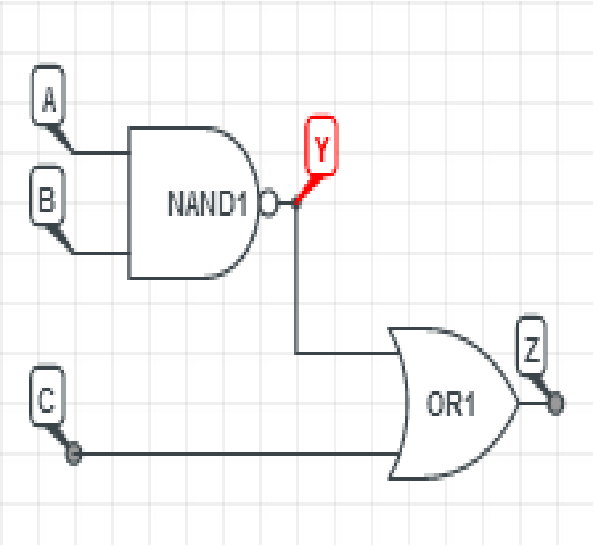
- **WAIT UNTIL**
  - Syntax: **`WAIT UNTIL signal condition;`**
  - Explanation: Accepts one signal, waits until signal condition before going to next statement in the process.
  - Example: **`WAIT UNTIL (clk'EVENT and clk='1');`**
  - Synthesizable: Yes (Can use in **synthesizable code** and **testbenches**)




```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY flipflop IS
        PORT (  D, Clock : IN  STD_LOGIC ;
                Q        : OUT STD_LOGIC) ;
END flipflop ;
ARCHITECTURE Behavior_2 OF flipflop IS
BEGIN
        PROCESS
        BEGIN
           WAIT UNTIL Clock'EVENT AND Clock = '1' ;
           Q <= D ;
        END PROCESS ;
END Behavior_2 ;
```

Wait Statements

- **WAIT ON**
  - Syntax: **`WAIT ON signal1 [,signal2,…];`**
  - Explanation: Accepts multiple signals, waits until any of the signals change.
  - Example: **`WAIT ON clk,rst;`**
  - Synthesizable: Yes (Can use in **synthesizable code** and **testbenches**)




```
SELECTOR: process
Variable Y: STD_LOGIC;
begin
    Y:= A NAND B;
    Z<= Y OR C;
    WAIT ON A,B,C;
end process;
```

Wait Statements (TESTBENCHES)

- **WAIT FOR**
  - Syntax: `WAIT FOR time`
  - Explanation: Wait for a certain amount of time before going to next statement in process.
  - Synthesizable: **No (Can use in testbenches only)**

- **WAIT**
  - Syntax: `WAIT`
  - Explanation: Waits forever, i.e. never goes to the next statement in process.
  - Synthesizable: **No (Can use in testbenches only)**

Conclusion

- Use processes with very simple structure only to describe
  - **Registers**
  - **Shift registers**
  - **Counters**
  - **State machines (next lecture)**
- Use examples discussed in the lecture as a template.
- Create **generic entities for registers, shift registers, and counters**, and instantiate the corresponding components in a higher level circuit using **GENERIC MAP PORT MAP**.
- Supplement sequential components with combinational logic described using concurrent statements

Conclusion

1. Sensitivity list of the PROCESS should include **only** signals that can by themselves change the outputs of the sequential circuit (**typically, clock and asynchronous set or reset**)
2. Do not use Processes without sensitivity list

Conclusion

Given a single signal, the assignments to this signal should only be made within a single process block in order to avoid possible conflicts in assigning values to this signal.

```
Process 1: PROCESS (a, b)
BEGIN
   y <= a AND b;
END PROCESS;

Process 2: PROCESS (a, b)
BEGIN
   y <= a OR b;
END PROCESS;
```



STATEMENTS WITHIN A PROCESS ARE SEQUENTIAL... BUT TWO PROCESSES ARE CONCURRENT TO ONE ANOTHER. THEY RUN IN PARALLEL